

Pipeline de Carcascor

1. Recorte de Loquetas

Partiendo de la fotografía del usuario con el fin de generar los recortes de cada loqueta con la biblioteca OpenCV, se le solicita al usuario marcar 8 loquetas que posteriormente servirán de guía para que el sistema interpole de manera acertada el resto de loquetas.



Ejemplo de una selección de loquetas.

2. Identificación de Componentes

Una vez obtenidos los recortes individuales de las loquetas, se procede a la etapa de identificación. En esta fase se determinan y clasifican los siguientes elementos.

2.1. Tipo de Loqueta

Para identificar cada una de las loquetas del tablero, se emplea una red neuronal convolucional (CNN)¹ y en caso de que la confianza de la detección sea menor al 70% se realiza un procedimiento basado en técnicas de procesamiento de imágenes utilizando la biblioteca OpenCV. Esto permite aumentar o disminuir la confianza en el caso de que los resultados sean iguales o difieran respectivamente. Luego, si la confianza es menor al 75% se le solicita al usuario que elija entre los dos

tipos más probables según el modelo para garantizar que la decisión final sea correcta. La necesidad de incorporar esta etapa de validación surge del análisis del *Top-2 Accuracy*, el cual mostró que, aún cuando el modelo no alcanza un nivel alto de confianza, la clase correcta suele encontrarse entre las dos predicciones más probables.



Ejemplo de verificación entre 2 loquetas más probables.

2.2. Meeple

Se detecta la presencia, la posición del meepo y el jugador al que pertenece. La localización del meepo se estima utilizando OpenCV dividiendo la loqueta en una grilla de 3x3. Cada celda de la grilla corresponde a una de las posiciones posibles, numeradas del 1 al 9.

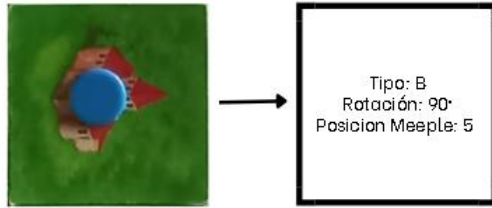
2.3. Rotación

La orientación de la loqueta se determina mediante métodos de análisis de imagen implementados con OpenCV. Se define una orientación estándar (sin rotación) y se identifican las rotaciones comparando las características visuales de la loqueta con sus patrones en cada ángulo. Siendo los ángulos posibles de 0° 90°, 180° y 270° desde la rotación estándar.

3. Origin Matrix

A partir de la identificación de los componentes de la loqueta, se genera una abstracción simple de datos denominada *origin matrix* que será la base para las dos fases de conteo de puntos.

1. Ver anexo CNN.

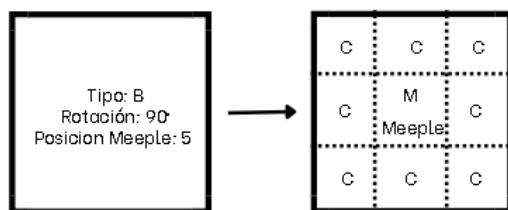


Ejemplo de la representación de una loseta en la *origin matrix*.

4. Conteo de Puntos

4.1. Castillos, caminos y monasterios

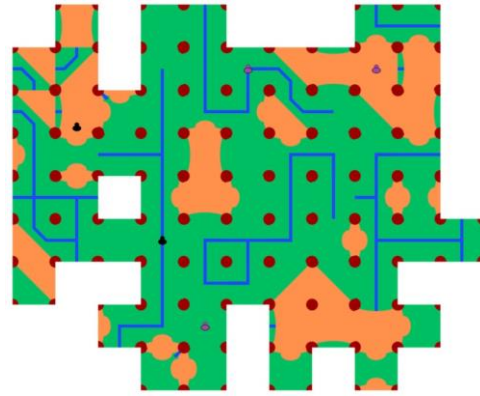
Se construye una abstracción derivada de la *origin matrix*. Esta abstracción especifica los tipos de terrenos que están presentes en cada loseta (campo, castillo, camino o monasterio) y, en caso de existir un meeple, identifica el terreno en el que se encuentra ubicado. Con esta información es posible identificar las conexiones entre losetas adyacentes y segmentarlas en unidades estructurales de cada terreno, a partir de las cuales se calcula la puntuación correspondiente.



Ejemplo de la representación de una loseta teniendo en cuenta sus terrenos (M = monasterio, C = campo).

4.2. Campos

Para calcular los puntos asociados a los campos, se genera una imagen a partir de la *origin matrix* para identificar las regiones de campo desde una perspectiva global. En la imagen se sustituyen las texturas originales por colores planos para facilitar el conteo de puntos con OpenCV.

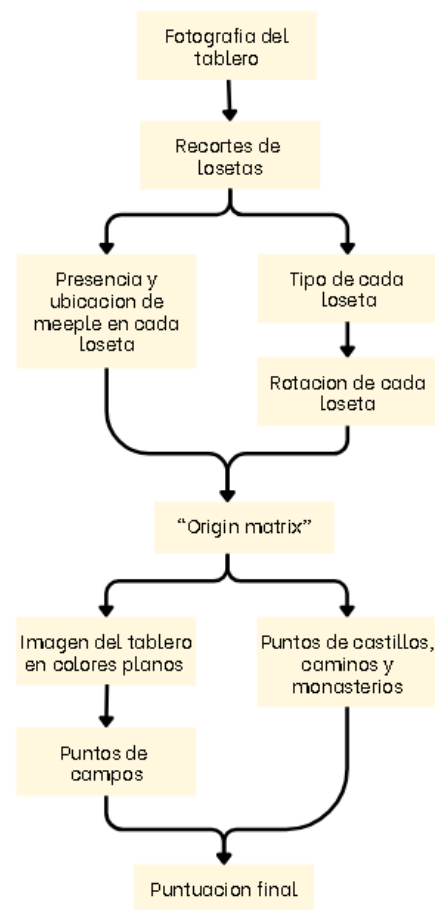


Ejemplo de una imagen generada a partir de la *origin matrix* para el conteo de los puntos de campos.

5. Resultado final

El resultado final es la suma de ambas fases del cálculo de puntos.

Diagrama del Pipeline



Anexo: Red Neuronal Convolucional

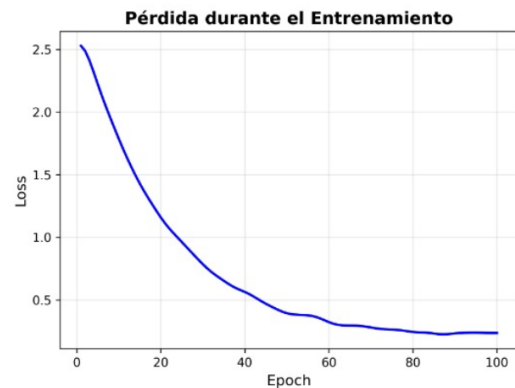
A partir de una imagen RGB de 64×64 píxeles, se utiliza una red neuronal convolucional para identificar el tipo de loseta de entre 25 clases. La arquitectura consta de cuatro capas convolucionales con 32, 64, 128 y 256 filtros que permiten aprender la información visual de forma jerárquica: primero bordes y contrastes, luego formas simples, después estructuras más complejas, y finalmente combinaciones de características específicas de cada loseta. Después de cada convolución se aplica un *Max Pooling* 2×2 , reduciendo el tamaño espacial ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$) y conservando sólo la información relevante.

El modelo cuenta con dos capas fully connected y se aplica Dropout del 50% luego de la primera para evitar que el modelo memorice los ejemplos de entrenamiento y mejorar su capacidad de generalización.

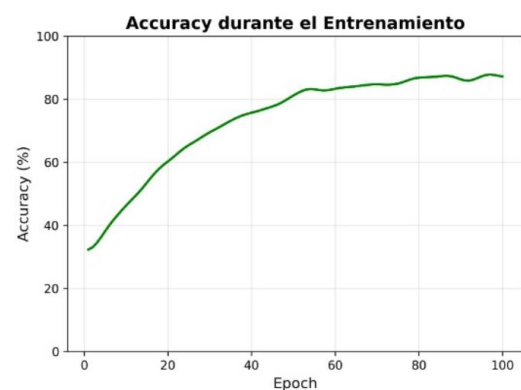
Para el entrenamiento se utilizaron 100 epochs, un batch size de 8 y la función de pérdida CrossEntropyLoss, estándar para clasificación multiclase. También se aplicó data augmentation (rotación, flips y variación de brillo/contraste), generando nuevas versiones de cada loseta para mejorar la robustez del modelo y su capacidad de generalización.

Durante el entrenamiento, el modelo mostró una mejora estable y consistente. El *Accuracy* alcanzó un valor final del 87.24%, mientras que la *Top-2 Accuracy* final fue de 93.98%, lo que significa que en casi el 94% de los casos la clase correcta se encuentra entre las dos predicciones más

probables. La *Loss* descendió rápidamente en las primeras etapas y se mantuvo alrededor de 0.25, indicando que el modelo continúa aprendiendo sin evidencia de sobreajuste.



Análisis de la *Loss*.



Análisis del *Accuracy*.

Para validar el desempeño real del modelo se realizaron pruebas manuales utilizando recortes reales de losetas. En estos tests, el modelo fue capaz de generalizar en la mayoría de los casos correctamente sobre imágenes que no formaban parte del dataset de entrenamiento.